Vericoding: Verification-Driven Development from Behavioural Features to Machine-Checked Proofs

The specsaver Toolchain

Gavin Mendel-Gleason
Scidonia

July 2026

Abstract

LLM-driven code generation ("vibecoding") trades correctness for velocity: generated code may run, but it leaves no durable specification, no behavioural guarantee, and no machine-checkable evidence that it satisfies its intended purpose. We propose *vericoding*, a verification-driven methodology in which a human-authored specification is the single source of semantic truth and the primary lasting artifact, with generated code as a replaceable implementation detail. The specification takes the form of Gherkin features (scenario tables), a mathematical contract (pure, boolean-valued predicates over pre/post-state, frames, and exceptions), and an abstract state schema; from this single specification the framework derives executable scenario checks, fine-grained telemetry assertions, and machine-checked proof obligations in Rocq (Coq) over a weakest-precondition calculus. Domain authors declare a *domain* once (DDL, row types, contracts, features); the framework supplies scenario runners, semantic frame checking, and differentially-tested stubs for library *theories* (SQL, logging, OpenTelemetry). A per-obligation scoreboard of PROVED, UNKNOWN, and DISPROVED entries provides a graded feedback cycle from test through proof to counterexample. The approach is battle-tested: the core contract language and theory machinery were extracted from a production citation-checking platform, and the worked examples include a production-stolen invitation flow that exercises insertion frames, multi-table deltas, and time-as-an-argument expiry.

1 Introduction

A growing fraction of production code is generated by large language models. The resulting programs often "work" in the sense that they execute without crashing, but they leave nothing behind: no specification of intended behaviour, no evidence of correctness, no reproducible conformance regime for the next iteration of the generated code. We call this pattern *vibecoding*: the developer prompts, the model emits, the result runs — and the only long-term artifact is the code itself, with no human-readable statement of what the code is supposed to do.

Vericoding inverts this relationship. In vericoding, the *specification* is the primary artifact and the code is a replaceable implementation detail. The specification is human-written and human-readable: Gherkin scenarios that domain experts can review, mathematical contracts that state what must hold before and after each operation, and framed state declarations that bound exactly what each operation may read and write. From this single specification every downstream verification activity is derived: concrete scenario execution against real database state, structured telemetry checking, and machine-checked proof obligations in Rocq (Coq).

specsaver is a toolchain for vericoding. The core thesis:

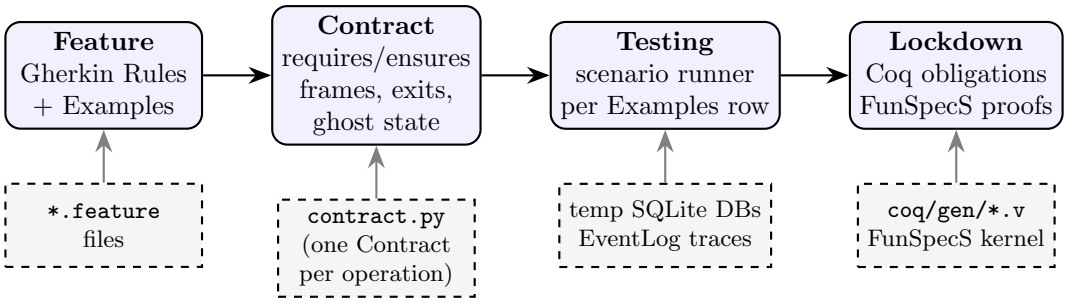

Figure 1: The vericoding pipeline. Features and contracts are the primary lasting artifacts; generated code (not shown) is a replaceable implementation detail. Every Examples row is exercised against the contract; the same contract lowers to Coq obligations proved against a hand-verified kernel.

Principle 1 (Single source of semantic truth)**.** *A human-authored specification, written once in the contract language, is simultaneously: (1) an executable behavioural specification checked against every row of a Gherkin Examples table, providing concrete pass/fail feedback in development; (2) a generator of verification artefacts (witness materialisation, admissibility checks, frame soundness, derived-state consistency, telemetry assertions); and (3) the input to a lowering that produces machine-checked proof obligations about an abstract model of the implementation — the "formalised lockdown" that sits behind the runtime checks.*

The pipeline of fig. 1 summarises the development flow: behavioural intent enters as *features*; features and implementation are jointly constrained by *contracts*; contracts drive *testing* against every row of every Examples table; and finally the same contracts are *lowered* to Rocq and proved — the "formalised lockdown."

The contributions of this paper are:

(i) a *contract language* that is the pure, terminating, boolean-valued fragment of Python, supporting pre/post-conditions, conjunctive exception exits, invariants, derived-state consistency, and *semantic frame conditions* (section 3);

(ii) a *symmetric projection architecture* connecting abstract specification state to concrete SQLite state through SQLAlchemy, so the same snapshot function serves as both pre- and post-state (section 4);

(iii) a *theory mechanism* that adorns real libraries (SQLAlchemy, Python logging, OpenTelemetry) with operational semantics and differentially-tested stubs (section 6);

(iv) a *domain declaration* that collapses per-domain infrastructure to a single object (section 7);

(v) a *lowering* to Rocq proof obligations over a small WP-calculus kernel, with all four example contracts' obligations machine-proved; plus a three-way scoreboard of PROVED/UNKNOWN/DISPROVED that covers the full spectrum from proof to counterexample (section 8).

2 Background and Motivation

The core failure mode of vibecoding is not that the code is wrong — much of it works — but that the developer loses any ability to answer "what is this code supposed to do?" once the generation session ends. The specification is ephemeral (the prompt), the behavioural checks are absent (no test suite), and the evidence of correctness is anecdotal ("it worked once"). When the next LLM rewrites the function, nothing ties the new version to the old guarantee.

These faults are more acute in database-backed services, where silent corruptions — partial writes across transaction boundaries, bystander rows mutated outside the declared frame, telemetry that diverged from committed state — may be invisible for months and are costly to prevent by review alone. The vericoding answer is to leave the contract in place and treat the code as replaceable: the contract is checked on concrete data today (so the guarantees are grounded in executable reality) and lowered to a formal model tomorrow (so the guarantees acquire mathematical force).

A second motivation is *library semantics*. Real services call real libraries — SQLAlchemy for persistence, Python logging for telemetry, OpenTelemetry for tracing. Any verification story that ignores these calls is a toy. We therefore need stubs with provable fidelity: replacements for the real libraries that behave identically on the fragment the contract language covers, and loudly reject anything outside it.

3 The Contract Language

A specsaver contract is a first-class Python object built by the `Contract` constructor (or the `@contract` decorator) in what we call the *fluid contract* style [1, 2]: the contract is a declarative value assembled from named clauses, not a set of imperative assertions scattered throughout the code. Figure 2 shows its anatomy.

The contract language is the **pure terminating fragment of Python**: every clause is a lambda whose body may contain only arithmetic, comparison, boolean operators, attribute access, and a small fixed set of combinator calls (`extends_by_one`, `implies`, `safe_div`). Side effects, loops, exceptions, and non-boolean returns are rejected by a static purity checker before the clause is executed or lowered. Every clause must evaluate to a Python `bool`: contracts reason about truth of propositions, not about computations.

Definition 1 (Contract)**.** *A contract for an operation op is a record* $\langle \Sigma, \mathit{args}, \mathsf{pre}, \mathsf{post}, \mathcal{X}, \mathcal{I}, \mathcal{D}, \mathcal{W}, \mathcal{R}, \mathcal{G} \rangle$ *where:*

- Σ *is the specification state type (observed, derived, ghost components);*
- *args is the typed argument record (a frozen dataclass);*
- $\mathsf{pre}(\sigma, a)$ *is a boolean predicate on the pre-state and arguments (admissibility);*
- $\mathsf{post}(\sigma, a, r, \sigma')$ *is a boolean predicate on pre-state, arguments, result, and post-state (success obligations);*
- $\mathcal{X} = \{ \langle E_i, C_i, W_i, P_i \rangle \mid i \in [1..n] \}$ *is the set of exceptional exits, each naming a raised exception type* E_i*, a conjunctive when-condition* C_i*, an exit-specific write frame* W_i*, and exit-specific post-conditions* P_i*;*
- $\mathcal{I}$ *is the set of invariants* $I(\sigma)$ *that must hold in every observable state;*
- $\mathcal{D}$ *maps derived field names to pure functions of observed state, checked for consistency at every snapshot;*
- $\mathcal{W}$ *and* $\mathcal{R}$ *are the write and read frame sets: declarative paths into* Σ *that bound what the operation may modify and observe;*
- $\mathcal{G}$ *is the ghost-state machinery: a ghost type* G*, an initialisation* $\mathsf{init}(w)$ *from the scenario witness* w*, a set of ghost transitions* $T(\gamma, a, r, \gamma')$*, and ghost invariants* $I_G(\gamma)$*.*

At the concrete level, each contract clause is a pure, terminating Python lambda over the fixed argument forms (σ, a)*,* (σ, a, r, σ')*, or* (σ, a, e) *(for exception payloads), statically checked to be boolean-valued and free of side effects before execution or lowering.*

3.1 Semantic frames

Frame conditions in classical DbC are notoriously verbose: specifying what *does not* change typically dominates the specification. specsaver frames are instead *declarative write paths*:

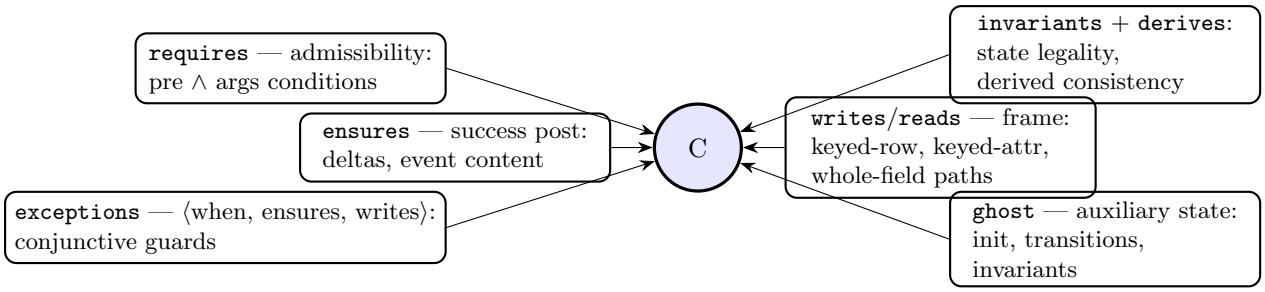

Figure 2: Anatomy of a contract. Each clause is a pure Python lambda; all clauses are executable, checkable, and (for the covered fragment) translatable.

```
writes={
    "state.products[sku].reserved", # one attribute of one keyed row
    "state.invitations[token]", # whole-row insert of the
                                # args-resolved key
    "state.reservation_log", # whole event channel
}
```

The checker derives, from these paths alone, the obligation that everything else is unchanged: every other keyed attribute, every other keyed row, every other event channel. A recent refinement admits *insertion*: keyed-row write paths may *add* the args-resolved key (e.g. a fresh invitation), while deletion is always rejected. This is essential for operations such as `invite` that create rows.

3.2 Exceptional exits

Exceptions are specified as data, not control flow:

```
exceptions=[
    ExcExit(
        raises=InsufficientStockError,
        when=[lambda state, args:
                  state.observed.products[args.sku].on_hand
                - state.observed.products[args.sku].reserved
                < args.quantity],
        writes={"state.failure_log"},
        ensures=[lambda state, args, exc, after_s: extends_by_one(
            state.observed.failure_log,
            after_s.observed.failure_log,
            lambda f: f.sku == args.sku
                      and f.available == exc.available
                      and f.reason == InsufficientStockError.code)],
    )]
```

The runner checks that when the `when` clauses hold, the exception raised is the declared one, that the failure telemetry contains exactly one new event with the exact declared fields, and that nothing outside the exit's writes changed. The classical "exceptions leave state untouched" discipline falls out of the default empty frame.

3.3 Ghost state

Ghost state is auxiliary specification state invisible to the implementation — e.g. the total stock present before the scenario, used to state conservation laws that are not expressible over the

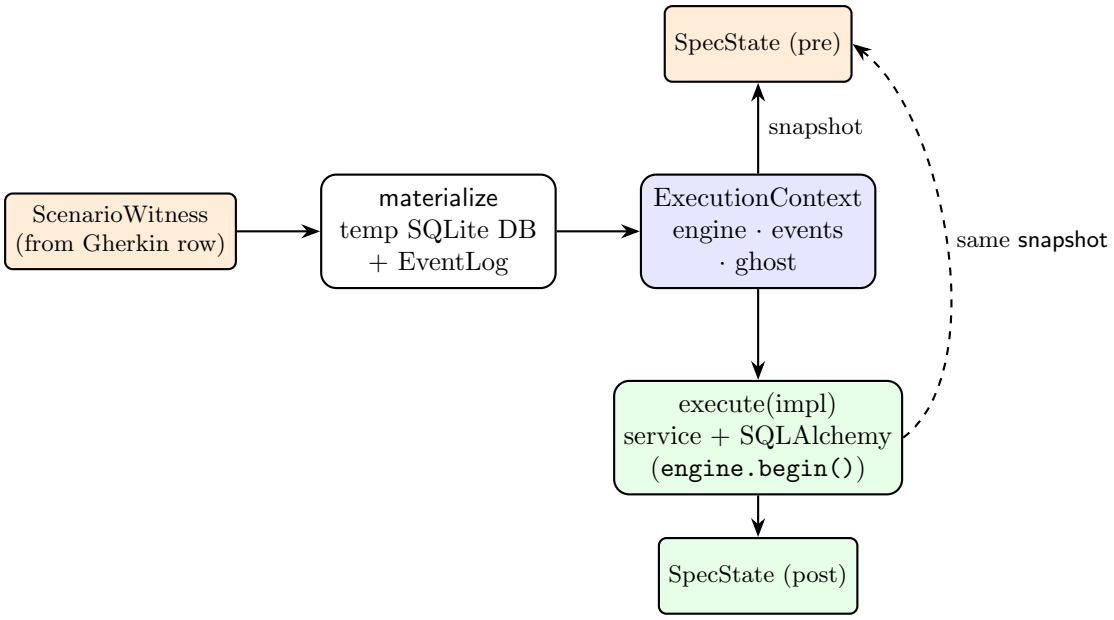

Figure 3: The symmetric projection. snapshot(context) projects the concrete world into the abstract SpecState both before and after execution; contracts compare pre and post. The service talks only to SQLAlchemy; the same service code runs on a real SQLite file and on the theory stub (section 6).

concrete DB alone. Ghost state is initialised from the scenario witness, constrained by transition relations per call, and held to global ghost invariants.

4 Symmetric Projection Architecture

The central architectural problem is the bridge between the abstract specification state (a frozen dataclass of observed, derived, and ghost components) and the concrete execution world (a live SQLite database plus a telemetry log). specsaver's answer is *symmetry*: one function, used twice.

Definition 2 (Symmetry requirement)**.** *Let* snap : $Ctx \to SpecState$ *be the projection. For every scenario, the runner evaluates all contract clauses against* $pre = \mathsf{snap}(c)$ *and* $post = \mathsf{snap}(impl(c))$*, for the same* snap*. There is no separate "read" and "check" path.*

The scenario runner then performs, per Examples row:

1. **materialize** the witness into a context (temp SQLite file);
2. **pre-check**: invariants on the pre-state, then admissibility (`requires`) — `rejected` outcomes are expected to fail admissibility, `error:...` outcomes to pass it;
3. **execute** the effect-emitting wrapper around the service;
4. **frame check**: everything outside the declared writes unchanged (for success) or the exit's writes (for exceptions);
5. **derived check**: derived fields recomputed equal their `derives` definitions;
6. **post-check**: `ensures` or the matching `ExcExit.ensures`; invariants on the post-state.

Crucially, the service is ordinary production code: SQLAlchemy `engine.begin()` transactions, `text()` statements, domain exceptions. Nothing in it knows about contracts, snapshots, or scenario rows.

5 Examples with Databases

We present three domains of increasing structural interest, all against SQLite through SQLAlchemy.

5.1 Inventory: deltas and conditional telemetry

The inventory domain models stock reservations over a single `products` table. The reserve contract's essential clauses, written in the mathematical style they realise:

$$\mathsf{pre}(s, a) \;=\; a.quantity > 0 \;\wedge\; a.sku \in s.products$$

$$\begin{aligned}\mathsf{post}(s, a, r, s') \;=\;& s'.products[a.sku].reserved \;=\; s.products[a.sku].reserved + a.quantity \\ &\wedge\; \mathsf{extends\_by\_one}(s.reservation\_log,\, s'.reservation\_log,\, \lambda e.\; e.sku = a.sku) \\ &\wedge\; \mathsf{implies}(crossed(s, a, s'),\; \mathsf{extends\_by\_one}(s.alert\_log,\, s'.alert\_log,\, \lambda a.\; a.sku = a.sku))\end{aligned}$$

$$\mathcal{X} = \{\, \langle \mathsf{InsufficientStock},\; s.available(a.sku) < a.quantity,\; \{failure\_log\},\; \mathsf{failure\_ensures}\rangle \,\}$$

with $\mathcal{W} = \{\, products[sku].reserved,\;\; reservation\_log,\;\; gauge\_log,\;\; alert\_log \,\}$. The frame checker derives, from $\mathcal{W}$ alone, that every other state path is unchanged: on-hand, reorder point, all other product rows, and the release/restock/failure channels. The invariant $\forall p \in products.\; 0 \le p.reserved \le p.on\_hand$ is checked in both pre- and post-state.

The Gherkin source rows are ordinary scenario outlines; the domain declares a witness builder mapping each row into an initial `products` dict and a typed `ReserveArgs`.

5.2 Bank transfer: multi-key deltas and invariants

The transfer contract moves funds between two rows of one table, with a balance-nonnegativity invariant and currency-mismatch and insufficient-funds exits. Its writes are two keyed attributes on *different* keys — demonstrating that frames compose over multi-delta operations — and its ensures include a conservation law (total balance preserved) and a negative-balance legality post-state invariant. The obligation generator lowers this to a two-key nested insert with conjunctive exception arms; all 7 obligations prove.

5.3 Invitations: insertion, multi-table state, and time

The invitations domain, stolen from a production service (paperchecker), exercises the newest machinery:

$$\begin{aligned}\mathsf{post}_{invite}(s, a, r, s') \;=\;& s'.invitations[a.token].status = \text{“pending”} \\ &\wedge\; s'.invitations[a.token].expires\_at \;=\; a.now + 7{\cdot}86400 \\ &\wedge\; \mathsf{extends\_by\_one}(s.invite\_log,\, s'.invite\_log,\, \lambda e.\; e.invitee\_email = a.invitee\_email)\end{aligned}$$

with $\mathcal{W}_{invite} = \{\, invitations[token],\; invite\_log \,\}$

$$\mathcal{X}_{accept} = \{\, \langle \mathsf{InvitationExpired},\; pending(s, a) \wedge a.now > s.invitations[a.token].expires\_at,\; \{accept\_failure\_log\},\; ..$$

$$\begin{aligned}\mathsf{post}_{accept}(s, a, r, s') \;=\;& s'.invitations[a.token].status = \text{“accepted”} \\ &\wedge\; s'.members[a.user\_id].org\_id \;=\; s.invitations[a.token].org\_id \\ &\wedge\; s'.members[a.user\_id].role \;=\; s.invitations[a.token].role \\ &\wedge\; s'.users[a.user\_id].default\_org \;=\; s.invitations[a.token].org\_id \\ &\wedge\; \mathsf{extends\_by\_one}(s.accept\_log,\, s'.accept\_log,\, \lambda e.\; e.user\_id = a.user\_id)\end{aligned}$$

Domain	Tables	Operations	Feature rows	Frame shape
inventory	1	reserve, release, restock	22	keyed-attr deltas
bank_transfer	2	transfer	11	multi-key deltas
invitations	3	invite, accept	9	insert + multi-table

Table 1: The three database-backed example domains. All feature rows are executed and checked by the generic conformance suite.

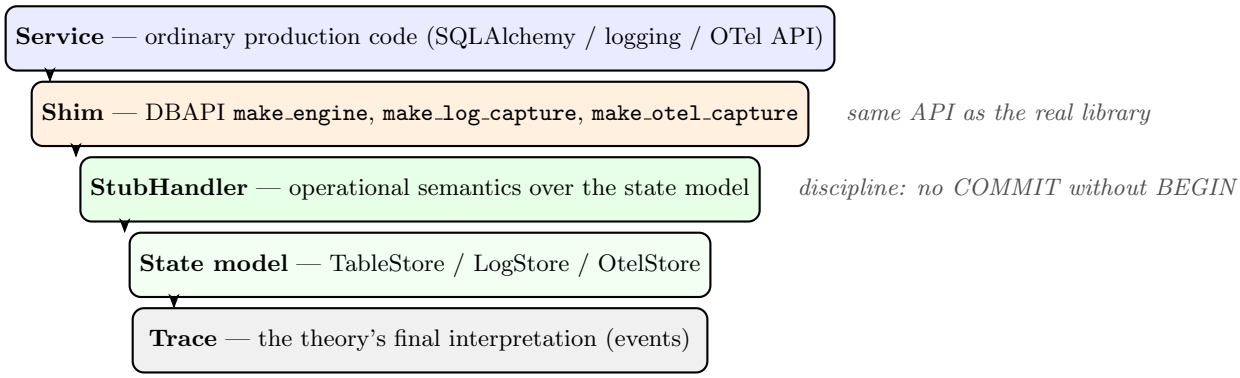

Figure 4: The theory stack. The service is written once, against the real library API; the shim connects that API to the stub; the stub interprets actions over a pure state model and records the trace. Differential tests run the same programs against stub and real library and require identical observable behaviour.

with $\mathcal{W}_{accept} = \{\, invitations[token].status,\ members[user\_id],\ users[user\_id].default\_org,\ accept\_log \,\}$.

Two design decisions make this contract-first: the accept-link token is a caller-supplied argument (so the insert key is args-resolved and the frame can see it), and `now` is an integer epoch argument (so expiry is a pure comparison, translatable and deterministic). Accepting touches three tables; the frame still derives that the fourth logical region — the email channel, user emails, every other invitation — is untouched.

6 Theories: Adorning Libraries with Semantics

Verification of library-calling code requires a stance on what the library does. specsaver's stance is the *theory*: a first-order operational model of the library's covered fragment, paired with a stub that behaves identically to the real library on that fragment — and rejects everything else loudly.

6.1 The SQL theory

The SQL theory covers a deliberately small fragment: equality-WHERE `SELECT` (with optional `ORDER BY` ascending), single-row `INSERT`, and `UPDATE` with `SET col = col ± expr`. Statements are parsed by `sqlglot` into an action model; anything outside the fragment raises `UnsupportedStatementError`.

The stub handler interprets the action model over an immutable `TableStore`, records the trace (`Begin`, `Execute`, `FetchOne`, `Commit`, `Rollback`), and enforces transaction discipline. Crucially, the same handler is exposed through a PEP-249 DBAPI shim, so *unmodified SQLAlchemy service code runs on the theory.* The sqlite dialect's deferred-autobegin semantics is modelled at the connection level; `commit()` and `rollback()` are, as in real sqlite3, no-ops without an open transaction, while the SQL-string path keeps strict discipline.

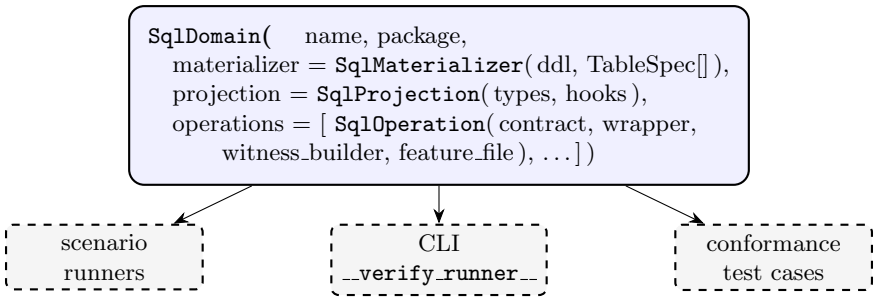

Figure 5: One `SqlDomain` declaration derives runners, CLI dispatch, and the conformance suite. The contract's `observe` is wired to the shared projection automatically (breaking the contract–projection import cycle).

Fidelity is established differentially: a battery of programs (SELECT/UPDATE/INSERT/BEGIN/COMMIT/ROLLBACK sequences) run both against real in-memory SQLite and against SQLAlchemy-over-stub, and fetches, final table contents, error classes, and traces must agree.

6.2 Logging and OpenTelemetry theories

The same six-layer shape repeats. For **logging**, the action model is a `LogEmit` (level, logger name, message, extra dict); the stub is a `logging.Handler` subclass; the capture context manager installs it at DEBUG level and restores state on exit; the store offers `by_level`/`by_logger` facet views. Differential fidelity compares against a real `StreamHandler`.

For **OpenTelemetry**, the action model is a `SpanRecord` (name, ids, attributes, events, status, timestamps); the stub is an `opentelemetry.sdk.trace.SpanProcessor`; the capture context manager builds a fresh `TracerProvider` with the stub attached (resetting OTel's one-set guard so sequential captures in a test process work). Differential fidelity compares against `InMemorySpanExporter`.

Principle 2 (Theory conformance)**.** *A theory is admissible only if: (1) its stub and the real library agree observationally on a differential suite; (2) its translator rejects out-of-fragment input loudly, never silently; (3) its state model and trace are pure data. New theories (e.g. an HTTP client, a queue) follow the same checklist.*

7 Domains: Declaring a Verification Unit Once

Experience with three domains revealed a stable shape: only types, service, contract, features, and witness builders are genuinely domain content; everything else (event log, temp-DB lifecycle, projection mechanics, runner wiring, test dispatch) was byte-identical infrastructure copied per domain. The `SqlDomain` declaration (fig. 5) collapses the latter.

A `TableSpec` names a table's columns, primary key, and the witness attribute carrying its initial rows; from these the materializer generates DDL execution, `DELETE`+ `INSERT` seeding, engine construction, and teardown. The projection queries each declared table and passes row dicts plus the event log to two small domain hooks (`extract_observed`, `compute_derived`). Domains with irregular tables (the key/value `limits` table in bank transfer) declare a `populate_extra` hook and query such tables manually inside `extract_observed`.

The conformance suite iterates every registered domain, every operation, and every Examples row, running the full check pipeline of section 4. Adding a domain is now: types, service, contracts, features, witness builders, one declaration — with the conformance guarantee that malformed pieces fail loudly at import time rather than silently at scenario time.

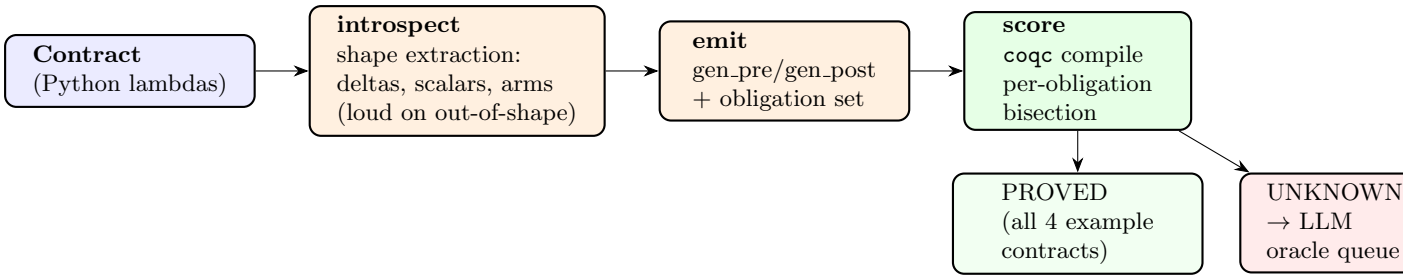

Figure 6: The lowering pipeline. The harness compiles whole-file, then bisects per obligation on failure, yielding a PROVED/UNKNOWN scoreboard; UNKNOWNs are packaged as self-contained proof problems for the oracle queue.

8 Lowering to Proof Obligations

The final stage is the "formalised lockdown": the same contract is introspected and emitted as a Rocq file of proof obligations over a small, hand-verified kernel.

8.1 The kernel

The Coq side comprises three layers. **SnakeletExn** is a tiny imperative language with dictionaries, lists, exceptions, and an evaluation relation. A **weakest-precondition calculus** over it (SnakeletExnWp) provides the standard structural tactics. **FunSpecS** is a specification kernel: a function table maps operation names to stateful specs $(pre, post)$ where *post* constrains the result and the list of heap-cell updates; a library of proven support lemmas (dictionary lookup/insert, store invariants) lives in `SpecPrelude`.

8.2 The generator

`specsaver.lower` introspects a `Contract` into a *shape*: typed row fields, per-key deltas, scalar preconditions, conjunctive exception arms, and the invariant's comparison structure. Anything outside the supported shape raises `UnsupportedShapeError` rather than emitting a wrong proof. The emitter then produces a file containing, per operation: `gen_pre/gen_post` (and per-arm exception pre/post), a function table, totality lemmas, invariant-preservation lemmas, and the obligation set — admissibility sanity, spec consistency, per-exit consistency, exit coverage, invariant preservation, and frame soundness (fig. 6).

An example fragment of generated code for reserve:

```
Definition gen_post (sigma : sn_state) (vs : list sn_val)
    (r : Result) (ups : cell_updates) : Prop :=
  exists sku order quantity store_d on_hand_sku reserved_sku
         reorder_point_sku,
    vs = [LitString sku; LitString order; LitInt quantity] /\
    sigma !! store_loc = Some (LitDict store_d) /\
    dict_lookup_str sku store_d =
      Some (row_of on_hand_sku reserved_sku reorder_point_sku) /\
    r = RVal (LitInt reserved_sku) /\
    ups = [(store_loc, LitDict (dict_insert_str sku
           (row_of on_hand_sku (reserved_sku + quantity)
                  reorder_point_sku) store_d))].
```

All four example contracts currently generate *and prove*: reserve (6/6 obligations), release (6/6), restock (4/4), transfer (7/7), including frame soundness and invariant preservation. Trace-cell obligations (list-append post-states for event channels) are the active development front (section 9).

The per-obligation scoreboard distinguishes three statuses. PROVED is the final word: the machine has checked the obligation against the kernel and closed it. UNKNOWN means the obligation did not close within the proof heuristics; it is queued for the LLM-based oracle. DISPROVED is a third status not yet wired into the harness but natural in our setting: the scenario runner already exercises every contract clause against concrete data from the Gherkin tables, and a failing runtime check for a well-formed contract is a counterexample — concrete evidence that the specification is inconsistent. Surfacing this as a first-class scoreboard entry closes the loop: proofs tell you what must be true; counterexamples tell you what cannot be true.

9 Related and Future Work

Design by Contract (Eiffel and successors) provides runtime assertions without a path to mechanised proof; specsaver keeps the executable-contract ergonomics and adds the lowering. **Property-based testing** (Hypothesis, QuickCheck) generates data but keeps properties informal; our contracts are simultaneously test oracles and proof obligations. **KeY, Frama-C, and Dafny** verify source-level contracts but require the source language to be the specification language; we instead verify an abstract model aligned with the contract's declared frame, keeping production code unmodified. **Separation-logic-based verification of library-calling code** typically models libraries by axiomatisation; our theories are *tested* against the real libraries, replacing axioms by evidence.

Current work: (i) trace-cell obligations in the lowering — event channels as list cells with `extends_by_one` as list-append post-conditions (the runtime side already enforces these); (ii) incremental re-verification keyed on contract/body hashes; (iii) loop and sequence support in the obligation generator; (iv) surfacing DISPROVED counterexamples from the runner into the scoreboard.

10 Conclusion

A vericoding workflow offers a concrete alternative to the vibecoding status quo. Where vibecoding leaves code and hope, vericoding leaves a specification and evidence: human-readable contracts that domain experts can review, concrete scenario checks that catch regressions before deployment, and machine-checked proofs that establish universal properties about the abstract model. The generated code is free to change — move from raw sqlite3 to SQLAlchemy, add a caching layer, replay events into a different schema — so long as it continues to satisfy the same contract. The specification survives.

Three ingredients make this practical: the symmetric projection that grounds contracts in real database state; the theory mechanism that replaces axiomatic library models with differentially-tested stubs; and the domain declaration that reduces a new verification unit to its irreducible content. The examples — reservation, transfer, and a production-stolen invitation flow from a real citation-checking platform — show that the idiom scales to multi-table, multi-exit, insert-bearing systems. The PROVED/UNKNOWN/DISPROVED scoreboard completes the feedback cycle: every row of every feature is a test; every contract is a proof obligation; and every runtime failure of a well-formed contract is a counterexample ready to be surfaced.

References

[1] Bertrand Meyer. *Applying "Design by Contract"*. IEEE Computer, 25(10):40–51, 1992.

[2] K. Rustan M. Leino and Peter Müller. *Using the Spec# Language, Methodology, and Tools to Write Bug-Free Programs*. In LASER Summer School 2008/2009, LNCS 6029, Springer, 2010.

[3] Gavin Mendel-Gleason. *specsaver: Specification-Driven Verification*. Source code and documentation. `https://github.com/scidonia/specsaver`, 2025–2026.

[4] Cucumber Ltd. *Gherkin Reference*. `https://cucumber.io/docs/gherkin/reference/`, 2025.

[5] Koen Claessen and John Hughes. *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs*. In ICFP 2000.

[6] John C. Reynolds. *Separation Logic: A Logic for Shared Mutable Data Structures*. In LICS 2002.

[7] Scidonia. *SnakeletExn and FunSpecS: A Small Imperative Language and Weakest-Precondition Specification Kernel*. Coq development, 2026.